

The Prompt Ladder

Baseline Prompt:

Help me write a case study for my Secure RAG project.

Output:

Before I help you write it — let me ask the right questions first.

A case study is not a project description. It tells a story of thinking and decision-making that a recruiter can follow.

The structure I'd recommend:

Problem → Approach → Technical Decisions → Results → Lessons

Let's build it from your own words so it sounds like you, not a template.

First question:

In one or two sentences — what was the core problem SecureRAG was designed to solve, and why does that problem matter to an enterprise?

Answer in your own words, not from your project README.

Notes:

What changed?

- This was my original simple prompt without any guidance.

What improved?

- The AI generated a basic case study.

What still failed?

- The response was generic and lacked technical depth. It did not reflect my personal experience.

What next?

- I will make the goal clearer.

Version 1:

Write a professional case study for my Secure RAG project that clearly explains the problem, my solution, and the outcome.

Output:

Secure RAG — Enterprise AI Document Intelligence

A RAG pipeline designed to help employees retrieve information from internal documents quickly, with security controls built for enterprise environments.

RAG ChromaDB Sentence Transformers PromptSecurity Python REST API

Problem

Employees waste time searching scattered documents

Organizations store knowledge across multiple documents — policies, reports, manuals — but employees have no efficient way to query them. This is a well-documented productivity problem in enterprise AI literature, and RAG pipelines have emerged as a practical solution.

"I identified this problem through technical blogs and case studies from companies building AI assistants for internal knowledge management — not through direct user interviews. SecureRAG is a technical demonstration of one possible solution, not a validated product."

Approach

Two users, two sets of requirements

I designed the system around two distinct users: employees who need fast, accurate answers from internal documents, and administrators responsible for security and compliance. Those users drove specific technical decisions — not just feature additions.

For employees — semantic retrieval

Used ChromaDB and Sentence Transformers to enable semantic search across documents. Employees can ask natural language questions instead of manually scanning files.

For administrators — layered security

Implemented prompt injection detection, toxic content filtering, query validation, and audit logging. Every interaction is monitored and traceable.

Key technical decision

Rule-based prompt injection detection — a deliberate tradeoff

The first security check normalizes user input, then compares it against a predefined list of injection patterns — phrases like "ignore previous instructions" or "reveal the system prompt." Detected requests are blocked before reaching the retrieval or generation stages, and the event is logged with a timestamp and security status.

I chose this approach because it is simple, transparent, and easy to test. It can be bypassed by rephrased attacks — I never considered it a complete solution. For this portfolio project, demonstrating the full security workflow (detect → block → log) mattered more than building an impenetrable system. In production, I would layer this with an LLM-based or ML classifier to catch semantic injection attacks that don't rely on fixed keywords.

Outcome

A realistic enterprise prototype

Status

Portfolio prototype

Security layers

3 (injection, toxicity, validation)

Retrieval method

Semantic (vector search)

Audit logging

Yes — every query

Lessons

What I learned

Security in AI systems isn't a feature you add at the end — it needs to be the first layer in the pipeline, before retrieval or generation.

Simple, explainable solutions have real value. A rule-based approach is easier to maintain, audit, and debug than a black-box model — especially in enterprise contexts where transparency matters.

Designing for two distinct users (employees and administrators) changed the architecture. Without that framing, I would have built a retrieval system — not an enterprise-grade one.

Notes

What changed in the prompt?

Added a clear goal.

What improved in the output?

The AI generated an actual case study instead of asking clarification questions.

What still failed?

The response was still generic and did not consider a specific audience.

What I would try next

Add the target audience.

Version 2:

Write a professional case study for my SecureRAG project that clearly explains the problem, my solution, and the outcome.

The audience is AI recruiters hiring entry-level AI Engineers.

Output

SecureRAG — Enterprise AI document intelligence

A production-style RAG pipeline that lets employees query internal documents using natural language, with enterprise security built into the pipeline from the start.

RAGChromaDBSentence TransformersPrompt securityPythonREST API

Problem

Employees waste hours searching scattered internal documents

Organizations store critical knowledge across policies, reports, and manuals — but employees have no efficient way to query them. They manually scan multiple files to find a single answer. This is a well-documented productivity problem in enterprise AI, and RAG pipelines have emerged as the practical solution.

Solution

A two-user system: fast retrieval for employees, security controls for admins

I designed SecureRAG around two distinct users with different needs, which drove the entire architecture.

For employees — semantic retrieval

ChromaDB and Sentence Transformers power vector search across uploaded documents. Employees ask natural language questions and get accurate answers without manual file scanning.

For admins — layered security pipeline

Every query passes through prompt injection detection, toxic content filtering, and query validation before reaching retrieval or generation. Every interaction is logged with a timestamp and security status.

The security check runs first — before retrieval, before generation. Detected threats are blocked and logged. Clean queries proceed through the RAG pipeline to produce a grounded answer.

Key technical tradeoff

Rule-based injection detection — simple by design

Prompt injection detection normalizes input, then matches it against a pattern list of known attack phrases ("ignore previous instructions", "reveal the system prompt"). Matched queries are blocked immediately.

This approach can be bypassed by rephrased attacks — I didn't design it to be the only defense layer. For this project, the goal was to demonstrate the full security workflow clearly: detect → block → log. In production, I'd combine this with an LLM-based or ML classifier to catch semantic injection that doesn't rely on fixed keywords.

Outcome

A realistic enterprise prototype

Type

Portfolio prototype

Security layers

3 (inject, toxicity, validation)

Retrieval

Semantic vector search

Audit logging

Every query

What I learned

Security belongs at the start of the pipeline, not bolted on at the end. Putting the security check before retrieval changed how I thought about the whole architecture.

Designing for two users — employees and admins — forced tradeoffs that made the system more realistic than a generic RAG demo would have been.

Explainable, auditable solutions have real enterprise value. A rule-based approach is easier to maintain and debug than a black-box model — especially where compliance matters.

Notes

What changed in the prompt?

Added a specific audience.

What improved in the output?

The AI focused more on my engineering skills instead of giving a general explanation.

What still failed?

The response still lacked detailed project-specific context.

What I would try next

Provide project context.

Version 3:

Write a professional case study for my SecureRAG project.

The audience is AI recruiters hiring entry-level AI Engineers.

Project Context:

SecureRAG is an enterprise document intelligence system built using Retrieval-Augmented Generation (RAG), ChromaDB, Sentence Transformers, Claude API, prompt injection detection, toxic content filtering, query validation, and audit logging.

Output

SecureRAG — Enterprise AI document intelligence

An enterprise-grade RAG system that lets employees query internal documents through natural language, with a security pipeline built in from the ground up.

RAGChromaDBSentence TransformersClaude APIPrompt securityPython

Problem

Employees waste hours searching scattered internal documents

Most enterprise knowledge is buried across policies, reports, and manuals with no efficient way to query it. Employees manually scan multiple files to find a single answer — a well-documented productivity problem that RAG pipelines are built to solve.

Solution

A secure, end-to-end RAG pipeline

Every query passes through a layered pipeline before a response is generated.

Security layer — runs first

Prompt injection detection, toxic content filtering, and query validation block unsafe requests before they reach retrieval or generation. Every interaction is logged with timestamp and security status.

Retrieval layer — semantic search

ChromaDB and Sentence Transformers embed documents and retrieve the most relevant chunks via vector search, replacing manual file scanning.

Generation layer — grounded answers

Claude API generates answers grounded in retrieved document content, reducing hallucination risk.

Outcome

Type

Portfolio prototype

Security checks

3 (inject, toxicity, validation)

Retrieval

Semantic vector search

Audit logging

Every query

What I learned

Security belongs at the start of the pipeline — putting it before retrieval shaped the entire architecture.

Building for two users (employees and admins) forced real tradeoffs that made this more than a generic RAG demo.

Explainable, auditable systems have genuine enterprise value — especially where compliance is a requirement.

Notes

What changed in the prompt?

Added project-specific context.

What improved in the output?

The AI produced a much more accurate case study using my actual project details.

What still failed?

The writing still sounded slightly like generic AI-generated content.

What I would try next

Specify my writing style.

Version 4:

Write in a direct, honest, practical, and professional tone.

Avoid buzzwords.

Only use information supported by my project.

Output

SecureRAG — Enterprise AI document intelligence

A portfolio prototype demonstrating how to build a secure, production-style RAG system for enterprise document retrieval.

RAGChromaDBSentence TransformersClaude APIPrompt securityPython

Problem

Finding information in internal documents takes too long

Employees in document-heavy organizations spend significant time searching across scattered files — policies, reports, manuals — to answer questions that should take

seconds. I studied this problem through enterprise AI case studies and technical documentation before designing a solution.

This is a research-informed prototype, not a deployed product. The problem is real; my solution is one practical approach to it.

Solution

Security first, then retrieval, then generation

I structured the pipeline so security runs before anything else. A query that fails the security check never reaches the retrieval or generation stages.

Security layer

Prompt injection detection (pattern matching on normalized input), toxic content filtering, and query validation. Every interaction is logged with a timestamp and security status for admin review.

Retrieval layer

ChromaDB stores document embeddings generated by Sentence Transformers. Queries are embedded and matched against documents using vector similarity search.

Generation layer

Claude API generates answers grounded in the retrieved document chunks, reducing the risk of hallucinated responses.

Honest assessment

What this project demonstrates — and what it doesn't

The rule-based injection detection can be bypassed by rephrased attacks. I chose it deliberately: it's transparent, easy to audit, and demonstrates the security workflow clearly. In a production system, I'd add an ML-based classifier to catch semantic injection attacks that don't match fixed patterns.

This system has not been used by real enterprise users. It shows I can design a system around user needs, make deliberate technical tradeoffs, and think beyond a basic demo.

What I learned

Architecture decisions change when you put security first instead of last.

Designing for two users — employees needing fast answers, admins needing traceability — produced a more realistic system than designing for one.

Knowing the limits of your own system matters as much as knowing what it can do.

Notes

What changed in the prompt?

Added quality requirements.

What improved in the output?

The response became more honest and matched my personal writing style.

What still failed?

Although the tone improved, the response became slightly longer and repetitive. Improving tone alone did not improve the structure.

Version 5:

Use the following structure:

Problem

What I Did

Outcome

Keep each section concise and under 120 words.

Output

SecureRAG — Enterprise AI document intelligence

Problem

Employees waste time searching internal documents manually

Organizations store knowledge across scattered files — policies, reports, manuals — with no efficient way to query them. Employees read through multiple documents to find a single answer. This is a documented productivity problem in enterprise environments that RAG pipelines are designed to address.

What I did

Built a secure, end-to-end RAG pipeline

I built SecureRAG using ChromaDB and Sentence Transformers for semantic document retrieval, and the Claude API for grounded answer generation. Before any query reaches retrieval, it passes through prompt injection detection, toxic content filtering, and query validation. Every interaction is logged with a timestamp and security status for admin review.

Outcome

A production-style prototype with honest tradeoffs

SecureRAG is a portfolio prototype — not a deployed product. It demonstrates I can design a system around real user needs, make deliberate architectural decisions, and build security into a pipeline from the start rather than adding it later.

Notes

What changed in the prompt?

Added a structured output format.

What improved in the output?

The response became much easier to read and looked like a professional portfolio case study.

What still failed?

Some sentences could still be refined manually before publishing.

What I would try next

Perform a final manual review before using the content.

Final Reusable Prompt:

Write a professional case study for my SecureRAG project.

Audience:

AI recruiters hiring entry-level AI Engineers.

Project Context:

SecureRAG is an enterprise AI document intelligence system built using Retrieval-Augmented Generation (RAG), ChromaDB, Claude API, Sentence Transformers, semantic search, prompt injection detection, toxic content filtering, query validation, and audit logging.

Write in a direct, honest, practical, and professional tone.

Avoid buzzwords.

Do not invent achievements or experiences.

Only use information supported by my project.

Use the following structure:

Problem

What I Did

Outcome

Keep each section concise, factual, and suitable for a professional AI portfolio.